Case Study 3: Bank Account and Money Transfers

Scenario

A retail bank wants a Java application that supports deposits, withdrawals and transfers. The current design has problems with overdraft limits, frozen accounts, incomplete transfers, incorrect transaction histories and invalid transaction amounts. The improved model uses entities, value objects, private data and operations that enforce banking rules.

Naive Model

```
class Account {\n    public String accountNumber;\n    public double balance;\n    public double overdraftLimit;\n    public boolean frozen;\n}\n\nclass TransferService {\n    void transfer(Account from, Account to, double amount) {\n        from.balance -= amount;\n        to.balance += amount;\n    }\n}
```

Problems:

- Public fields allow any class to change an account balance or status without checking business rules.
- Using double for money can cause rounding errors.
- There is no validation to reject zero or negative transaction amounts.
- The withdrawal method may not check the overdraft limit or account status.
- A transfer may update one account but fail before updating the other account.
- The account balance and transaction history can become inconsistent.

Q1. Domain Invariants

Business invariants are rules that must always remain true in the banking system.

1. Transaction amount must be positive.

- Rule: Every deposit, withdrawal and transfer amount must be greater than zero.
- Importance and consequence: It prevents meaningless or invalid transactions. If violated, the system may record zero-value or reverse-direction transactions.

2. Withdrawal must respect the overdraft limit.

- Rule: A withdrawal must not make the account balance lower than the permitted overdraft balance.
- Importance and consequence: It prevents customers from taking more money than the bank allows. If violated, the bank may face unauthorized credit risk.

3. Frozen accounts cannot be used for withdrawals or transfers.

- Rule: A frozen account must reject outgoing transactions until it is unfrozen.
- Importance and consequence: It protects accounts restricted for security, legal or administrative reasons. If violated, restricted money may be moved.

4. Closed accounts cannot perform transactions.

- Rule: A closed account must not accept deposits, withdrawals or transfers unless a specific bank policy allows a final adjustment.

- Importance and consequence: It prevents activity on an account that is no longer active. If violated, records and customer balances may become unreliable.

5. A transfer must be atomic.

- Rule: A transfer must debit the source and credit the destination together, or neither change must be committed.
- Importance and consequence: It prevents money from disappearing or being created. If violated, one account loses money while the other does not receive it.

6. Balance must agree with the ledger.

- Rule: The current balance must equal the opening balance plus all posted credits minus all posted debits.
- Importance and consequence: It provides an auditable and trustworthy account history. If violated, customers and bank reports may show different balances.

7. Every posted transaction must have a valid, unique record.

- Rule: Each completed transaction must have an identifier and must not be posted twice.
- Importance and consequence: It supports auditing and prevents duplicate processing, especially when a request is retried.

Q2. Entities and Value Objects

An entity has its own identity and can change over time. A value object is defined by its values, is usually immutable, and does not need a separate identity.

Concept Classification Justification

- Bank Account — Entity: It has a unique account identity and its balance and status change during its lifetime.
- Account Number — Value Object: It is identified by its validated value and is used to identify an account; it has no independent lifecycle.
- Money — Value Object: Two Money values with the same amount and currency are interchangeable; it should be immutable.
- Ledger Entry — Entity: Each posted entry has a unique transaction/entry ID and remains part of the permanent transaction history.
- Overdraft Limit — Value Object: It represents a limit amount, normally compared by its value and currency, rather than by identity.
- Fund Transfer — Entity: It has a unique transfer ID and a lifecycle such as requested, pending, completed or failed.

Note: A ledger entry can be modelled as an immutable value inside a ledger aggregate in some designs. It is classified as an entity here because the bank needs to identify, audit and refer to each individual posted entry.

Q3. Self-Enforcing Objects and Java Design

The Account class should protect its fields by making them private. Other classes must use public methods such as deposit() and withdraw() instead of changing the balance directly. Money should be an immutable value object stored in minor units (for example, paise) to avoid floating-point errors.

```
final class Money {\n    private final long paise;\n\n    private Money(long paise) { this.paise = paise; }\n\n    static Money ofRupees(long rupees) {\n        if (rupees < 0) throw new IllegalArgumentException();\n        return new Money(Math.multiplyExact(rupees, 100));\n    }\n\n
```

```
Money plus(Money other) {\n        return new Money(Math.addExact(paise, other.paise));\n    }\n\n
Money minus(Money other) {\n        return new Money(Math.subtractExact(paise, other.paise));\n
}\n\n    boolean isPositive() { return paise > 0; }\n    long inPaise() { return paise; }\n}
enum AccountStatus { ACTIVE, FROZEN, CLOSED }\n\nfinal class Account {\n    private final
AccountNumber number;\n    private Money balance;\n    private final Money overdraftLimit;\n
private AccountStatus status;\n\n    Account(AccountNumber number, Money openingBalance,\n
Money overdraftLimit) {\n        this.number = number;\n        this.balance = openingBalance;\n
this.overdraftLimit = overdraftLimit;\n        this.status = AccountStatus.ACTIVE;\n    }\n\n
public void deposit(Money amount) {\n        requireActive();\n        requirePositive(amount);\n
balance = balance.plus(amount);\n    }\n\n    public void withdraw(Money amount) {\n
requireActive();\n        requirePositive(amount);\n        Money newBalance =
balance.minus(amount);\n        if (newBalance.inPaise() < -overdraftLimit.inPaise())\n
throw new IllegalStateException("Overdraft limit exceeded");\n        balance = newBalance;\n
}\n\n    public void freeze() {\n        if (status == AccountStatus.CLOSED)\n            throw new
IllegalStateException("Account is closed");\n        status = AccountStatus.FROZEN;\n    }\n\n
public void unfreeze() {\n        if (status != AccountStatus.FROZEN)\n            throw new
IllegalStateException("Account is not frozen");\n        status = AccountStatus.ACTIVE;\n    }\n\n
public void close() {\n        if (status == AccountStatus.FROZEN)\n            throw new
IllegalStateException("Resolve freeze first");\n        if (balance.inPaise() != 0)\n
throw new IllegalStateException("Balance must be zero");\n        status = AccountStatus.CLOSED;\n
}\n\n    private void requireActive() {\n        if (status != AccountStatus.ACTIVE)\n
throw new IllegalStateException("Account is not active");\n    }\n\n    private void
requirePositive(Money amount) {\n        if (amount == null || !amount.isPositive())\n
throw new IllegalArgumentException("Amount must be positive");\n    }\n}
```

How the design enforces the rules:

- Invalid amounts are rejected by requirePositive() before the balance is changed.
- withdraw() checks that the account is ACTIVE and that the new balance is not below the negative overdraft limit.
- Frozen and closed accounts fail the requireActive() check, so withdrawals and deposits are rejected.
- Private fields prevent external classes from assigning a new balance or changing account status directly.
- In a production implementation, balance changes and ledger records must be saved together in one database transaction.

The code is an illustrative domain model. A real implementation should also validate currencies, use a validated AccountNumber value object, and persist changes and ledger entries atomically.

Q4. Money Transfer and Aggregate Consistency

1. Problems caused by the crash:

- Account A is debited by ₹5,000, but Account B is not credited.
- The customer loses access to the transferred amount even though the destination has not received it.
- The two account balances and their transaction histories may no longer represent a completed transfer.

2. Invariants violated:

- The transfer atomicity invariant is broken because only one side of the transfer was completed.
- The ledger-to-balance consistency invariant may be broken if the debit and credit records are incomplete.
- The transfer completion rule is broken because the transfer is only partially applied.

3. Design to prevent money loss:

For accounts managed by the same banking system and database, perform the debit, credit and both ledger postings inside one database transaction. The transaction commits only if every operation succeeds; otherwise, the database rolls back all changes.

```
BEGIN TRANSACTION\n  validate source and destination accounts\n  validate amount and available overdraft\n  debit Account A\n  credit Account B\n  write debit ledger entry\n  write credit ledger entry\nCOMMIT\n\nIf any step fails: ROLLBACK
```

4. Handling failures:

- If the transaction fails before commit, rollback leaves both account balances unchanged.
- Use a unique transfer ID or idempotency key so retrying the same request does not post it twice.
- Record transfer states such as PENDING, COMPLETED and FAILED, and reconcile pending transfers after a system restart.
- Use reliable transaction logging and monitoring so incomplete operations can be detected and resolved.

5. Transfers between different banking systems:

A single database transaction usually cannot cover two independent banks. Use a coordinated process such as a saga or a payment network protocol. The sending bank records a pending transfer, the receiving bank confirms the credit, and the transfer is marked complete only after confirmation. If the credit cannot be completed, the sending bank follows a defined reversal or compensation process. Messages should be durable, uniquely identified and safe to retry.

Q5. Ledger Management and Design Evaluation

1. Information in a ledger entry:

- Unique ledger entry ID and related transfer/transaction ID.
- Account number or account reference.
- Date and time of the transaction.
- Transaction type, such as deposit, withdrawal, transfer debit or transfer credit.
- Amount and currency.
- Debit or credit direction.
- Status, such as pending, posted or reversed.
- Reference or description, and the resulting balance where required by the bank.

2. Should a completed ledger entry be changed or deleted?

No. A posted ledger entry should be immutable and should not be deleted. If a transaction must be corrected, the bank should create a separate reversal or adjustment entry that refers to the original entry. This preserves the audit trail and shows exactly what happened.

3. Keeping the balance consistent with the history:

- Update the account balance and insert the corresponding ledger entries in the same database transaction.
- Use database constraints and unique transaction IDs to prevent incomplete or duplicate postings.
- Calculate or periodically verify the balance from the opening balance and posted ledger entries.
- Run reconciliation checks and investigate any difference between the stored balance and ledger total.

4. Challenges when the history becomes very large:

- Queries and reports may become slower as the number of entries increases.
- Storage and backup requirements grow over time.
- Searching and reconciling old transactions can require more processing.
- The bank may need indexing, partitioning by account or date, archival policies and efficient pagination.
- Archived records must still remain secure, retrievable and available for audits and legal requirements.

Illegal States Eliminated

- Zero and negative transaction amounts cannot be posted.
- Withdrawals cannot exceed the permitted overdraft limit.
- Frozen and closed accounts cannot perform prohibited transactions.
- A same-bank transfer either completes fully or has no committed effect.
- Ledger entries provide a permanent record, and corrections are made through reversals rather than deletion.
- Private account fields prevent uncontrolled changes from outside the Account class.

Discussion Points

- Money should be represented exactly, for example as integer paise, rather than double. The model should also ensure that amounts use the correct currency.
- For transfers between separate banks, a local database rollback is not enough. The design needs durable messaging, status tracking, idempotency and a compensation or reversal process.
- The ledger is the audit record. It should be append-only so that every original transaction and later correction remains visible.